

NATURAL LANGUAGE PROCESSING (NLP)

A series of horizontal lines in teal and light blue colors, extending from the left edge of the slide and ending under the title text.

Natural Language Processing

- Gives computers the ability to understand text and spoken words like human do.
- It combines statistical, machine learning and deep learning models to achieve this.
- Human text and voice data is broken down using various tasks as follows:

Speech recognition

- Also called speech-to-text.
- It is challenging as people may speak in different accents and intonations and often using incorrect grammar.

Part of speech tagging

- Also called grammatical tagging.
- It determines the part of speech (i.e., verb , noun etc.) of a particular word in a sentence.

Word of sense disambiguation

- Selection of the meaning of a particular word with multiple meanings that makes the most sense in the given context.

Named entity recognition (NER)

- Identifies words or phrases as useful entities.
- It identifies 'John' as a man's name and 'London' as a place.

Co-reference resolution

- Identifies if and when 2 words refer to the same entity.
- Eg: 'she' = 'Sally' or 'bear' as a large hairy person and not animal.

Sentiment analysis

- Extract subjective qualities-attitudes, emotions, confusion etc.

Natural language generation

- Text to speech

NLTK

- Natural Language Toolkit
- An open source collection of libraries, programs etc. for building NLP programs.
- Contains all tools for performing the mentioned tasks.

Text preprocessing-level 1

Tokenization

- A token is a part; like a word is a token in a sentence, and a sentence is a token in a paragraph.
- Tokenization is the splitting of a string into a list of tokens.

Stemming

- It is text standardization.
- Words are stemmed or diminished to their root.
- Eg: programmer, programming all stems to program.
- It can cause to loose the meaning of the root and the actual token.
- It only looks at the form of the word.

Lemmatization

- Lemmatization that ensures the word doesn't lose its meaning.
- It has a pre-defined dictionary that stores the context of words and checks the words in the dictionary while stemming.
- It looks at the meaning of the words.

Stop word removal

- Commonly used words that do not add any value to the analysis.
- Eg: I, me, myself, our, we, you etc.

POS tagging

- Parts of Speech tagging
 - Nouns: defines an object or entity
 - Verbs: defines actions
 - Adjectives and adverbs: acts as a modifier, quantifier or intensifier

Tag	Description	Tag	Description
1. ‘ ’	Quotation mark (“	20. RB	Adverb
2. ,	Comma	21. RBR	Adverb, comparative
3. .	Period (. ? !)	22. RBS	Adverb, superlative
4. :	Punctuation (: ; ... + - = < > / [] ~)	23. RP	Particle
5. CC	Coordinating conjunction	24. RT	Retweet
6. CD	Cardinal number	25. TO	<i>to</i>
7. DT	Determiner	26. UH	Interjection
8. EX	Existential <i>there</i>	27. URL	Universal Resource Locator
9. HT	Hashtag	28. USR	Username (preceded by @)
10. IN	Preposition or subordinating conjunction	29. VB	Verb, base form
11. JJ	Adjective	30. VBD	Verb, past tense
12. JJR	Adjective, comparative	31. VBG	Verb, gerund or present participle
13. JJS	Adjective, superlative	32. VBN	Verb, past participle
14. MD	Modal	33. VBP	Verb, non-3rd person singular present
15. NN	Noun, singular or mass	34. VBZ	Verb, 3rd person singular present
16. NNP	Proper noun, singular	35. WDT	Wh-determiner
17. NNS	Noun, plural	36. WP	Wh-pronoun
18. PRP	Personal pronoun	37. WRB	Wh-adverb
19. PRP\$	Possessive pronoun		

Chunking

- Process of extracting phrases from unstructured text.
- For example, it treats South Africa as a single word instead of treating South and Africa as separate words.
- It works on POS tagged data.
- Chunk tags :
 - Noun Phrase(NP)
 - Verb phrase (VP)
 - Adjective phrase (ADJP)
 - Adverb phrase (ADVP)
 - Prepositional phrase (PP)
- In order to create a chunker, first we have to define a chunk grammar.
- It contains the rules that indicate how sentences should be chunked.

- Chunk grammar is generated with the help of regular expressions.
- Commonly used regexp symbols and meanings are given below.

Symbol	Meaning
*	The preceding character can occur zero or more times meaning that the preceding character may or may not be there.
+	The preceding character should occur at least once.
?	The preceding character may not occur at all or occur only once meaning the preceding character is optional.

Named entity recognition(NER)

- Also called entity identification, entity extraction or entity chunking.
- It is the process of detecting the named entities such as person names, location names, company names etc. from the text.
- It automatically scans the text and group the fundamental entities into predefined categories. Entities may be:
 - Organizations,
 - Quantities,
 - Monetary values,
 - Percentages, and more.
 - People's names
 - Company names
 - Geographic locations (Both physical and political)
 - Product names
 - Dates and times
 - Amounts of money
 - Names of events

Regular Expressions

- Used for pattern matching in the text.
- To preprocess the format of texts in a document.
- To ensure that the first word in a sentence begins on uppercase.
- To find whether a sentence ends with a question mark and so on.

<https://www.h2kinfosys.com/blog/nltk-regular-expressions/>

Language models in NLP

- A language model is a probabilistic statistical model that determines the probability of a given sequence of words occurring in a sentence based on the previous words.
- Language models are of 2 types:
 - **Statistical language models:** Uses traditional methods to learn probability distribution of words
 - **Neural language models:** Uses neural networks.
- Used in machine translation, speech recognition, handwriting recognition, spelling correction etc.
- The input to a language model is a set of example sentences and the output is a probability distribution over sequence of words.

- The probability of each translation is computed to understand which one is the most accurate.
- Eg: $p(\text{the cat is small}) > p(\text{small the is cat})$
- The last one word (**unigram**), last two words (**bigram**), last three words (**trigram**) or last n words (**n-gram**) can be used to predict the next word.

N-gram language model (unigram, bigram, trigram)—Statistical model

- N-gram is a sequence of N tokens.

Eg: *“The ability to model the rules of a language as a probability gives great power for NLP related tasks”*

- A 1-gram (unigram) is a one-word sequence.

Here, the unigrams would be ‘The’, ‘ability’, ‘to’, ‘model’, etc.

- A 2-gram (bigram) is a two-word sequence of words.

‘The ability’, ‘ability to’, ‘to model’, ‘model the’ etc.

- A 3-gram (trigram) is a three-word sequence of words.

‘The ability to’, ‘ability to model’ etc.

How the model works



Predict $p(w|h)$ – what is the probability of seeing a word w given a history of previous words h .

When the history (context) of the word is approximated by considering only the last word of the context, the assumption is called the **Markov assumption**.

- Prediction of next words using trigram model
 - Here the probability of each new word depends on the previous two words

For example, the probability of the sentence '**I love dogs**' is ,

$P(\text{I love dogs}) = P(\text{I}) P(\text{love} \mid \text{I}) P(\text{dogs} \mid \text{I love})$ where,

$P(\text{I}) = \text{freq}(\text{I}) / \text{Total no. of words}$

$P(\text{love} \mid \text{I}) = \text{freq}(\text{I love}) / \text{freq}(\text{I})$

$P(\text{dogs} \mid \text{I love}) = \text{freq}(\text{I love dogs}) / \text{freq}(\text{I love})$

Here, $\text{freq}(\text{I})$, $\text{freq}(\text{I love})$ and $\text{freq}(\text{I love dogs})$ are the counts of the respective unigrams, bigrams and trigrams.

- Prediction of next words using bigram model
 - Probability of each new word depends only on its previous word.

$P(\text{I love dogs}) = P(\text{I}) P(\text{love} \mid \text{I}) P(\text{dogs} \mid \text{love})$

Limitations of ngram modeling

- The higher the n , the better the model, but higher will be the computational cost.
- It is based on probability of words co-occurring. If the words are not present in the training corpus, it will give zero probability to those words. If such ngrams occur in testing set, the overall probability of the test sentence becomes 0.

Text processing -level 2

Word Embeddings

- Language Processing by computers:
 - NLP (Natural Language Processing)
 - NLG (Natural Language Generation)
- Both NLP and NLG has increased the ability of computers to understand the text-based content.
- To understand and generate text, NLP systems should recognize words, grammar etc.
- Word embeddings map the words as real-values numerical vectors so that the NLP algorithms can easily digest it.
- It is done by tokenizing each word in a sequence and converting them into a vector space.

Word embedding techniques

Bag of Words (BoW)

- It represents the sentence as a bag of terms rather than its order and structure of the words.
- The algorithm transforms the text into fixed-length vectors.

Review 1: Game of Thrones is an amazing tv series!

Review 2: Game of Thrones is the best tv series!

Review 3: Game of Thrones is so great

	amazing	an	best	game	great	is	of	series	so	the	thrones	tv
0	1	1	0	1	0	1	1	1	0	0	1	1
1	0	0	1	1	0	1	1	1	0	1	1	1
2	0	0	0	1	1	1	1	0	1	0	1	0

TF-IDF (Term Frequency - Inverse Document Frequency)

- Based on statistical measure of finding the relevance of words in the text.
- TF score
 - Based on the frequency of words in a document.
 - Words are counted for their number of occurrences in the documents.

$$TF = \frac{\text{No. of occurrence of a word in the sentence}}{\text{No. of words in the sentence}}$$

- **IDF score**

- TF gives more weightage to words occurring more frequently.
- It gives an account of how unique a word is.
- But, words rarely used words may hold significant information. IDF captures this information.

$$IDF = \log \left(\frac{\text{No.of sentences}}{\text{No.of sentences containing the word}} \right)$$

$$\text{TF-IDF score} = \text{TF} * \text{IDF}$$

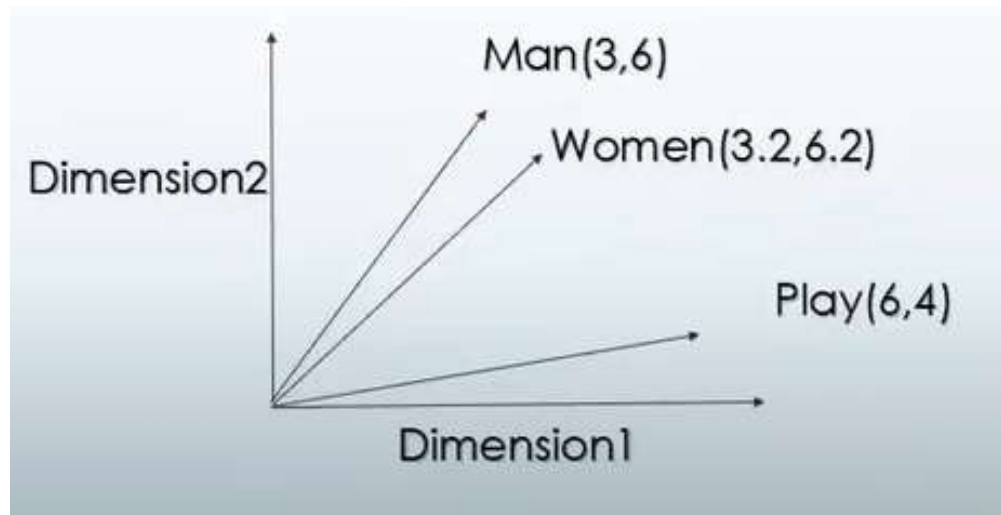
- The lower the IDF value of a word, the less unique it is to any particular document.
- Used in information retrieval, keyword extraction, stop words removal etc.
- 2 modules are available for this: Tfidftransformer and Tfidfvectorizer.
- $\text{Tfidfvectorizer} = \text{Count vectorizer} + \text{Tfidftransformer}$

BoW and TF-IDF problems

- Both of them do not store semantic information.
- TF-IDF can give more importance to uncommon words.

Word2Vec

- Captures the semantic information.
- Word embedding technique to solve advanced NLP problems.
- Here each word is represented as a vector of multiple dimension instead of a single number.
- It scans for similarities among words using the cosine similarity metric and helps in predicting a target word based on its context(neighboring words).



- If the cosine angle is 0, the words are overlapping.
- If the cosine angle is 90, the words are independent (no contextual similarity).
- 2 neural network- based architectures are used for this purpose:
- **CBOW**(Continuous Bag of Words) and **skip-gram**.
- CBOW uses the context to predict a target word.
- Skip Gram uses the current word to predict the neighboring words (context). This is more accurate on large datasets.

Neural language models

Recurrent Neural Networks (RNN)

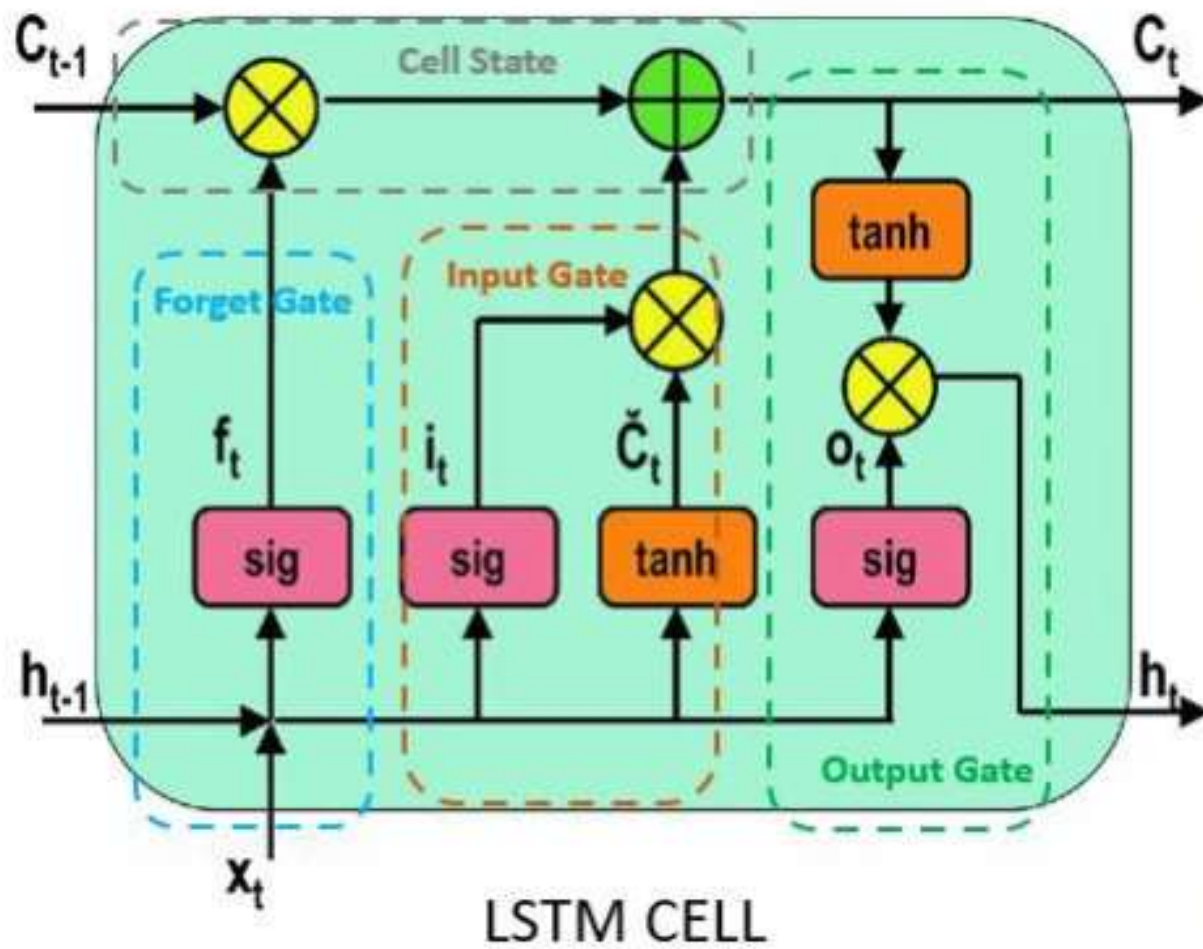
- NLP using traditional networks have certain limitations.
 - Prediction depends highly on the specific words.
 - Changing any word with its synonym can deviate the results.
 - Prediction was not dependent on the sequence of words.
- RNN solves the problem in which the sequence pattern is used for prediction.
- It is used for various applications like speech recognition, music synthesis, image captioning etc. where sequence of word matters.
- Problems with RNN: Exploding and vanishing gradient problem.
- This causes RNN unable to hold information of longer sequences leading to short-term memory.
- **LSTM** (Long-short-term memory) and **GRU** (Gated Recurrent Unit) solves this.

How LSTM and GRU help

Exploding gradient problem was due to capturing relevant and irrelevant information. LSTM and GRU have gates as an internal mechanism which control what information needs to be kept and what should be thrown out.

LSTM

- 3 gates to control the flow of information and cells to hold information.
- Cell states carry the information from initial to later time without getting vanished.
- Gates make use of sigmoid activation or tanh activation.
- 3 gates
 - Forget gate
 - Input gate
 - Output gate



sig = Sigmoid function

tanh = tanh function

 = point-by-point multiplication

 = point-by-point addition

 = vector connections

Forget gate

- Decides what information should be carried out forward or what should be ignored.
- Information from previous hidden states and the current state information passes through the sigmoid function.
- Values that come out from sigmoid are always between 0 and 1. if the value is closer to 1 means information should proceed forward and if value closer to 0 means information should be ignored.

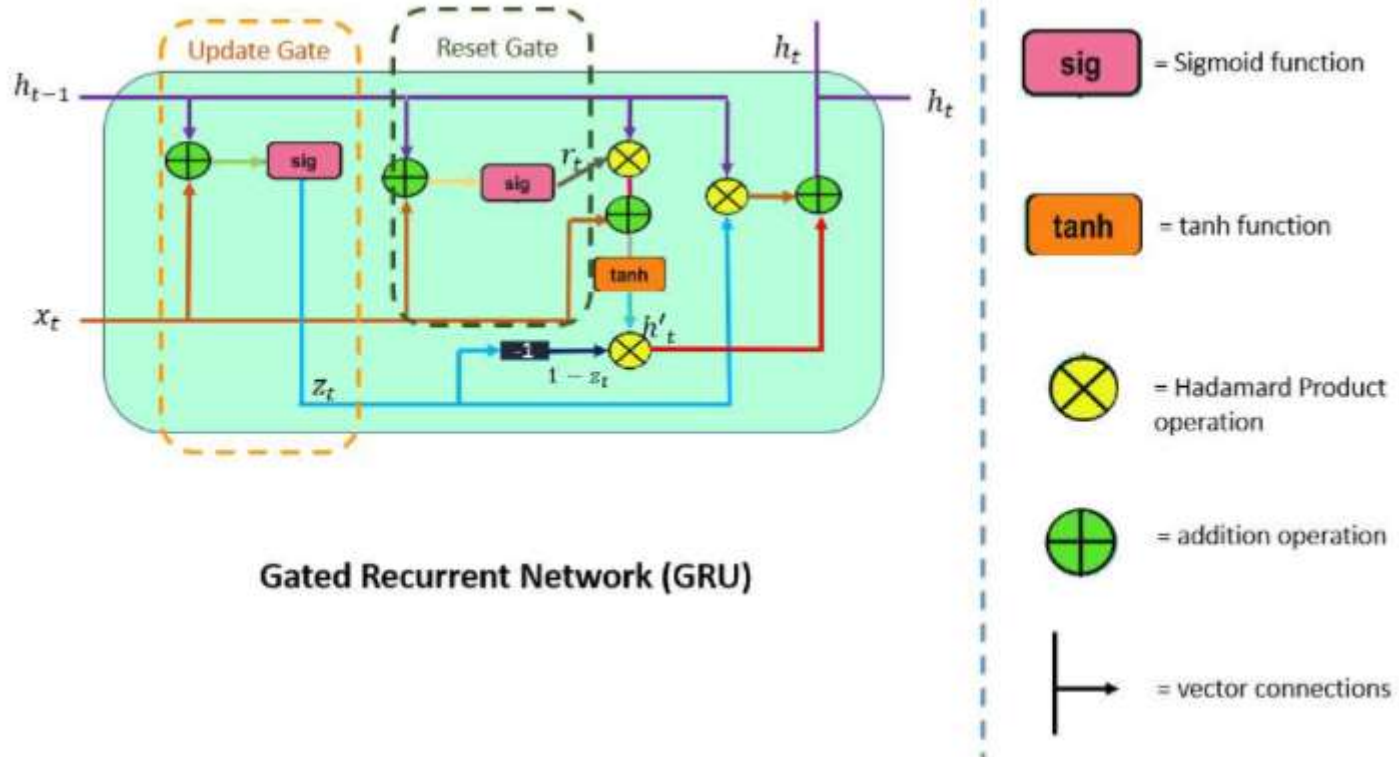
Input gate

- Input gate passes the relevant information which updates the cell states.
- New relevant information is added to the existing information by updating cell states.

Output gate

- Generates the next hidden states, and cell states are carried over the next time step.

GRU



- Similar to LSTM in some aspects.
- GRU is a modified, lighter version of LSTM
- It combines short-term and long term memory.
- An LSTM uses both a cell state and a hidden state.

- GRU doesn't contain a cell state
- It uses only its hidden state to transport information
- Only 2 gates: Reset and Update gate
- Faster than LSTM

Update gate

- Update Gate is a combination of Forget Gate and Input Gate.
- It determines how much past information should be retained to the future.

Reset gate

- Resets the past information in order to get rid of gradient explosion.
- It determines how much past information should be forgotten.